

Comparação de Algoritmos de Ordenação: Uma Análise de Desempenho Aplicada ao Sistema de uma Biblioteca Universitária

Adrian Pessoa Nery

Alexandre Cadeira Queiróz

Ana Carolina Dias de Souza Cornélio

Fernanda Figueiredo Silva Abreu

Gabriel Alves dos Santos

João Gonçalves Brandão Jácome

Maria Leticia Castanheira Odilon

Natália Lira Ferraz Novaes

Rayssa Kauany de Siqueira Alves

Universidade Federal do Vale do São Francisco – UNIVASF

Salgueiro
2026

RESUMO

A ordenação de dados é uma das operações fundamentais da Ciência da Computação, essencial para otimizar processos de busca e manipulação de informações. No contexto de sistemas que lidam com registros massivos, como o de uma grande biblioteca universitária, gargalos de desempenho frequentemente estão atrelados à ineficiência desses métodos. O presente trabalho tem como objetivo realizar uma análise comparativa entre os algoritmos Bubble Sort, Insertion Sort, Merge Sort e Quick Sort, avaliando sua adequação para resolver o problema de lentidão no gerenciamento de dados dessa biblioteca. A metodologia consistiu em pesquisa bibliográfica associada à análise experimental, medindo tempo de execução, quantidade de comparações e número de trocas em vetores de 1.000, 10.000 e 100.000 elementos, dispostos em estados ordenado, inversamente ordenado e aleatório. Os resultados empíricos demonstram que algoritmos de complexidade $O(n \log n)$, como Merge Sort e Quick Sort, apresentam desempenho significativamente superior em grandes volumes. Em contrapartida, algoritmos $O(n^2)$, como Bubble Sort e Insertion Sort, mostram severas limitações de escala, embora o Insertion Sort exiba eficiência em conjuntos previamente ordenados. Conclui-se que, para o volume de dados do sistema da biblioteca, a adoção de algoritmos baseados em divisão e conquista é a solução mais indicada e eficiente.

Palavras-chave: algoritmos de ordenação; complexidade computacional; análise de desempenho; estruturas de dados; eficiência algorítmica.

ABSTRACT

Data sorting is one of the fundamental operations of Computer Science, essential for optimizing information search and manipulation processes. In the context of systems that handle massive records, such as a large university library, performance bottlenecks are often linked to the inefficiency of these methods. This work aims to perform a comparative analysis between the Bubble Sort, Insertion Sort, Merge Sort, and Quick Sort algorithms, evaluating their suitability for solving the problem of slow data management in this library. The methodology consisted of bibliographic research associated with experimental analysis, measuring execution time, number of comparisons, and number of swaps in vectors of 1,000, 10,000, and 100,000 elements, arranged in sorted, inversely sorted, and random states. The empirical results demonstrate that algorithms with $O(n \log n)$ complexity, such as Merge Sort and Quick Sort, exhibit significantly superior performance in large volumes. In contrast, $O(n^2)$ algorithms, such as Bubble Sort and Insertion Sort, show severe scalability limitations, although Insertion Sort exhibits efficiency on previously sorted sets. It is concluded that, for the volume of data in the library system, the adoption of divide-and-conquer based algorithms is the most suitable and efficient solution.

Keywords: sorting algorithms; computational complexity; performance analysis; data structures; algorithmic efficiency.

1. INTRODUÇÃO

O desempenho de sistemas computacionais está diretamente relacionado à eficiência dos algoritmos utilizados em suas operações internas. Em aplicações que manipulam grandes volumes de dados, a escolha de métodos adequados para organizar informações torna-se um fator essencial para garantir rapidez, eficiência e qualidade na execução das tarefas. Nesse cenário, algoritmos de ordenação possuem papel fundamental, uma vez que são amplamente utilizados em sistemas que dependem da organização eficiente de registros para operações de busca, armazenamento e recuperação de dados.

No contexto deste trabalho, considera-se o cenário de uma biblioteca universitária que precisa lidar constantemente com grandes quantidades de registros, como títulos de livros, cadastros de alunos, datas de empréstimos e devoluções. Em ambientes como esse, a eficiência na organização desses dados impacta diretamente a agilidade no acesso às informações e, conseqüentemente, a qualidade dos serviços prestados aos usuários.

A literatura especializada, como destacam Ziviani (2007) e Cormen et al. (2012), evidencia que a escolha do algoritmo de ordenação mais adequado depende da análise rigorosa de sua complexidade teórica e de seu comportamento prático. Diante desse contexto, surge o seguinte problema de pesquisa: qual dos algoritmos analisados (Bubble Sort, Insertion Sort, Merge Sort ou Quick Sort), escolhidos por serem os mais comuns em disciplinas iniciais e por representarem diferentes classes de complexidade, apresenta a melhor adequação e eficiência para o sistema da biblioteca universitária, considerando variações no volume e na disposição inicial dos dados?

A partir desse problema, foram estabelecidas três hipóteses principais para orientar a análise experimental: (H1) em vetores de grande porte contendo dados em estado aleatório, os algoritmos Merge Sort e Quick Sort apresentarão desempenho significativamente superior ao Bubble Sort e ao Insertion Sort; (H2) em cenários nos quais os dados já se encontram previamente ordenados, o Insertion Sort apresentará desempenho altamente competitivo, realizando um número mínimo de operações; e (H3) em vetores com dados inversamente ordenados, Bubble Sort e Insertion Sort apresentarão desempenhos semelhantes e elevados em termos de custo operacional, refletindo o pior caso de sua complexidade $O(n^2)$.

Dessa forma, o objetivo deste estudo é realizar uma análise comparativa entre esses quatro algoritmos, considerando seus comportamentos teóricos e desempenhos práticos. Para isso, foram realizados experimentos utilizando vetores de números inteiros com 1.000, 10.000 e 100.000 elementos, em estados ordenado, inversamente ordenado e aleatório, avaliando métricas como tempo de execução, número de comparações e quantidade de trocas realizadas durante o processo, indicadores escolhidos por serem relevantes para mensurar o real custo computacional e o impacto no tempo de resposta do sistema da biblioteca.

2. FUNDAMENTAÇÃO TEÓRICA

Esta seção descreve os principais elementos teóricos a serem usados na análise e discussão empírica dos algoritmos de ordenação investigados na pesquisa.

2.1 Algoritmos de Ordenação e Complexidade Computacional

Entende-se por ordenação de dados o processo de rearranjar um conjunto de elementos em uma sequência lógica, seja ela crescente ou decrescente, visando otimizar a recuperação e o processamento de informações (SCHILDT, 2006). Segundo Ziviani (2007), a escolha de um algoritmo de ordenação adequado é fundamental na Ciência da Computação, pois impacta diretamente a eficiência de sistemas que lidam com grandes volumes de dados.

Para avaliar e comparar matematicamente a eficiência desses algoritmos, utiliza-se a análise de complexidade, frequentemente expressa por meio da notação Big-O. Cormen et al. (2012) complementam que a notação Big-O descreve o tempo de execução de um algoritmo no limite assintótico, ou seja, como o número de operações cresce em proporção ao tamanho da entrada de dados (n). No contexto deste estudo, os algoritmos dividem-se em duas categorias principais de complexidade de tempo: quadrática e logarítmica.

2.2 Algoritmos de Complexidade Quadrática

Schildt (2006) demonstra que algoritmos de complexidade $O(n^2)$ são de fácil implementação, mas apresentam severas limitações de desempenho quando aplicados a arranjos grandes.

- Bubble Sort: É um método baseado em sucessivas trocas de elementos adjacentes que estão fora de ordem. A cada passagem, o maior elemento "flutua" para o final do arranjo. Conforme Schildt (2006), embora algoritmos com essa complexidade sejam de fácil implementação, eles apresentam severas limitações de desempenho quando aplicados a arranjos grandes. Uma das razões para isso é que o Bubble Sort realiza diversas comparações e passagens desnecessárias mesmo quando os elementos já se encontram próximos da posição correta. Por conta disso, apesar de seu inegável valor didático para estudantes, o Bubble Sort possui tempo de execução $O(n^2)$ tanto no pior caso quanto no caso médio, tornando-se computacionalmente ineficiente para problemas de grande escala (CORMEN et al., 2012). Neste trabalho, o Bubble Sort foi implementado em sua versão clássica, sem a otimização de parada antecipada (flag de verificação de trocas).
- Insertion Sort: É um algoritmo de ordenação que organiza os elementos inserindo cada item em sua posição correta dentro da parte já ordenada do vetor. Seu funcionamento é semelhante à organização de cartas em um baralho, realizando comparações e deslocamentos até encontrar a posição adequada para cada elemento. Segundo Cormen et al. (2012), o algoritmo possui implementação simples, baixo consumo de memória e estabilidade, preservando a ordem de elementos iguais. Neste estudo, adotou-se a implementação clássica do Insertion Sort baseada estritamente em deslocamentos sucessivos dos elementos até a posição correta, sem otimizações adicionais. Sua complexidade é $O(n^2)$ no caso médio e no pior caso, mas pode alcançar desempenho $O(n)$ em listas já ordenadas ou parcialmente ordenadas (CORMEN et al., 2012). Além disso, estudos comparativos demonstram que o Insertion Sort pode apresentar desempenho prático superior ao Bubble Sort. Segundo Al-Amin, Okeyinka e Ibrahim (2024), isso ocorre porque o Insertion Sort tende a utilizar menos operações em cenários favoráveis, enquanto o Bubble Sort realiza comparações desnecessárias mesmo com os elementos quase na posição correta.

2.3 Algoritmos de Complexidade Logarítmica

Para cenários de alto volume de processamento, como o gerenciamento dos vastos registros da biblioteca universitária propostos, os algoritmos baseados na estratégia de

"divisão e conquista" tornam-se essenciais, alcançando complexidade na ordem de $O(n \log n)$.

- Merge Sort: O funcionamento do Merge Sort consiste em dividir repetidamente o array em duas partes até que cada subdivisão contenha apenas um elemento. Depois disso, o algoritmo começa a unir essas partes, comparando os elementos e organizando-os em ordem crescente ou decrescente, até que todos os elementos estejam reunidos novamente em um único array já ordenado (BACKES, 2016). Ele é uma excelente opção para ordenar grandes quantidades de dados porque mantém um desempenho eficiente, apresentando complexidade $O(n \log n)$ garantida (CORMEN et al., 2012). Além disso, Ziviani (2007) destaca que o Merge Sort é um algoritmo estável, preservando a ordem correta dos dados quando há mais de um critério de ordenação, como autor, nome do estudante e data de empréstimo, porém, vale ressaltar que ele necessita obrigatoriamente do uso de memória auxiliar para a execução do processo de fusão das subdivisões, fator que impacta diretamente suas limitações de espaço.
- Quick Sort: Seu funcionamento gira em torno de um elemento chamado pivô. A lógica consiste em particionar o array original em três partes: um subarray com os elementos menores que o pivô, o próprio pivô, e um subarray com os elementos maiores. Esse processo de divisão é aplicado de forma recursiva até chegar ao caso-base, que são arrays com zero ou um elemento (BHARGAVA, 2017). A complexidade do Quick Sort varia conforme a divisão do arranjo. Em seu caso médio e melhor caso, apresenta uma complexidade de tempo $O(n \log n)$, que ocorre quando o pivô divide o array em duas partes quase idênticas (CORMEN et al., 2012). No entanto, Bhargava (2017) ressalta que o pior caso ocorre quando o pivô escolhido é o primeiro ou o último elemento em uma lista já ordenada (ou inversamente ordenada), levando a uma complexidade de tempo $O(n^2)$. Nessa situação, a árvore de recursão torna-se degenerada, exigindo consideravelmente mais interações para ordenar o array. Neste trabalho, o Quick Sort foi implementado utilizando uma estratégia de pivô fixo definido como o último elemento do subvetor analisado, o que justifica a convergência para o pior caso teórico nos cenários estruturados.

3. METODOLOGIA

Os algoritmos selecionados para este estudo (Merge Sort, Quick Sort, Insertion Sort e Bubble Sort) foram definidos a partir da situação-problema proposta pelo orientador. O objetivo consistiu em analisar as características de cada um sob diferentes cenários de teste, avaliando o desempenho e o tempo de execução na ordenação de vetores com tamanhos variados. Todos os códigos utilizados no teste foram aproveitados da web (Geeks For Geeks) e alterados pelos autores.

3.1 Definição da Massa de Dados e Cenários de Teste

Para a realização dos experimentos e avaliação empírica do desempenho dos algoritmos selecionados, foi gerada uma massa de dados controlada composta por números inteiros. A geração, armazenamento e manipulação dessas entradas foram desenvolvidas utilizando a linguagem de programação C, aplicando a função *rand()* da biblioteca *stdlib.h* para a distribuição dos valores de forma automatizada.

Com o objetivo de analisar o comportamento e a eficiência das estruturas sob diferentes ordens de magnitude e escalas de complexidade, foram definidos três tamanhos distintos de vetores. As volumetrias de entrada estabelecidas para os testes foram:

- 1.000 elementos (pequena escala);
- 10.000 elementos (média escala);
- 100.000 elementos (grande escala).

Para cada um dos tamanhos de entrada mapeados, os dados foram organizados em três cenários estruturais distintos antes do início do processo de ordenação. Essa estratégia permitiu simular e avaliar as perspectivas teóricas de melhor caso, caso médio e pior caso de complexidade de cada algoritmo:

- Cenário Ordenado (Melhor Caso): Os elementos foram dispostos previamente em ordem estritamente crescente. Este cenário visa mensurar o custo computacional e a capacidade do algoritmo em identificar e validar uma estrutura de dados que já se encontra organizada, minimizando o número de trocas.
- Cenário Aleatório (Caso Médio): Os elementos foram distribuídos de forma desordenada e pseudoaleatória no vetor. Este cenário busca simular uma situação

real e cotidiana de uso, onde o estado inicial dos dados não possui um padrão previsível de organização.

- Cenário Inversamente Ordenado (Pior Caso): Os elementos foram gerados e posicionados em ordem estritamente decrescente. Este cenário foi desenhado para forçar os algoritmos ao seu limite superior de esforço, exigindo a quantidade máxima de operações de comparação e movimentação de dados permitida por suas respectivas lógicas estruturais.

Nota de Rigor: Para garantir a isonomia dos testes, não foi utilizada a função `srand(time(NULL))`. Ao manter a semente (*seed*) padrão do sistema fixa, a função `rand()` gerou exatamente a mesma sequência de números pseudoaleatórios para todos os algoritmos em cada execução. Desse modo, garantiu-se de forma nativa via código que todos os quatro algoritmos fossem submetidos a arranjos com os mesmos valores e ordens idênticas para cada cenário. Isso garantiu que nenhuma estrutura levasse vantagem por pegar um vetor “mais fácil” que as outras.

3.2 Ambiente de Desenvolvimento e Configuração de Hardware

Para garantir a reprodutibilidade dos testes, as execuções dos algoritmos foram realizadas sob as configurações de hardware e software descritas a seguir.

3.2.1 Configuração de Hardware

- Processador: Intel Core i3-1115G4 (11ª Gen) @ 3.00GHz (2 núcleos, 4 threads);
- Memória RAM: 20 GB;
- Armazenamento: SSD NVMe;
- Arquitetura: 64 bits.

3.2.2 Configuração de Software e Compilação

- Sistema Operacional: Windows 11 Home Single Language (64 bits);
- Linguagem de Programação: Linguagem C (padrão C99);
- Compilador: GCC;
- Flags de Compilação: Sem otimização (-O0 implícito), garantindo a medição do tempo de execução do código bruto de cada algoritmo.

3.3 Protocolo de Testes e Métricas Analisadas

Para garantir a precisão e a confiabilidade dos dados coletados, o procedimento experimental seguiu um rigoroso protocolo de controle de variáveis, sendo executado da seguinte forma:

- **Controle do Ambiente de Teste:** Todas as simulações foram executadas estritamente no mesmo dispositivo (descrito na Seção 3.2), com o mínimo de processos em segundo plano ativos. Essa unificação garantiu que flutuações de processamento do hardware não interferissem de forma desigual nos resultados dos diferentes algoritmos.
- **Métricas Coletadas:** Os códigos-fonte originais dos algoritmos foram instrumentados com variáveis contadoras para registrar três métricas fundamentais:
 - **Tempo de Execução:** Cronometrado em milissegundos utilizando as funções da biblioteca padrão *time.h* da linguagem C, capturando a diferença exata entre o início e o fim apenas do processo de ordenação (excluindo o tempo de geração ou leitura dos vetores).
 - **Número de Comparações:** Contabilização de quantas vezes dois elementos do vetor tiveram seus valores avaliados entre si durante a execução.
 - **Número de Trocas:** Registro de quantas vezes os elementos foram movidos ou trocaram de posição na memória para alcançar a ordenação.
- **Repetições e tratamento de dados:** Cada algoritmo foi executado 10 vezes consecutivas para cada combinação de tamanho (1.000, 10.000, 100.000) e cenário de ordenação (ordenado, aleatório e inverso). A cada rodada, as três métricas individuais foram registradas. Ao final das 10 repetições, foi calculada a média aritmética simples dos resultados para neutralizar eventuais anomalias do sistema operacional. Foram esses valores médios que serviram de base para a construção das tabelas, gráficos e posterior análise dos resultados.

4. RESULTADOS

Nesta seção são apresentados os resumos de todos os resultados obtidos neste trabalho através dos testes realizados.

4.1 Cenário de Pequena Escala (1.000 elementos)

Tabela 1A – Desempenho com Vetor Ordenado

Algoritmo	Tempo (ms)	Comparações	Trocas
Bubble Sort	1,401	$5,00 \times 10^5$	0
Insertion Sort	0,005	$9,99 \times 10^2$	0
Merge Sort	0,104	$5,04 \times 10^3$	$5,04 \times 10^3$
Quick Sort	2,737	$5,00 \times 10^5$	$5,00 \times 10^5$

Fonte: elaborado pelos autores (2026)

Tabela 1B – Desempenho com Vetor Inversamente Ordenado

Algoritmo	Tempo (ms)	Comparações	Trocas
Bubble Sort	2,621	$5,00 \times 10^5$	$5,00 \times 10^5$
Insertion Sort	1,735	$5,00 \times 10^5$	$5,00 \times 10^5$
Merge Sort	0,118	$4,93 \times 10^3$	$4,93 \times 10^3$
Quick Sort	1,874	$5,00 \times 10^5$	$2,50 \times 10^5$

Fonte: elaborado pelos autores (2026)

Tabela 1C – Desempenho com Vetor em Ordem Aleatória

Algoritmo	Tempo (ms)	Comparações	Trocas
Bubble Sort	2,595	$5,00 \times 10^5$	$2,45 \times 10^5$
Insertion Sort	0,901	$2,46 \times 10^5$	$2,45 \times 10^5$
Merge Sort	0,172	$8,70 \times 10^3$	$8,70 \times 10^3$
Quick Sort	0,093	$1,05 \times 10^4$	$6,23 \times 10^3$

Fonte: elaborado pelos autores (2026)

A análise conjunta dos resultados demonstra que, em um cenário de pequena escala com vetor ordenado, o Insertion Sort apresenta um rendimento superior aos demais, registrando o tempo de 0,005 ms e nenhuma troca. No entanto, o algoritmo perde eficiência nos estados inversamente ordenado (1,735 ms) e aleatório (0,901 ms). O Bubble Sort manteve tempos similares nos casos inverso e aleatório, com aproximadamente 2,60 ms. O Merge Sort demonstrou estabilidade em todos os cenários, situando-se na faixa entre 0,104 ms e 0,172 ms. O Quick Sort conquistou o menor tempo no estado aleatório (0,093 ms), porém apresentou os piores resultados nos casos ordenado (2,737 ms) e inverso (1,874 ms).

4.2 Cenário de Média Escala (10.000 elementos)

Tabela 2A – Desempenho com Vetor Ordenado

Algoritmo	Tempo (ms)	Comparações	Trocas
Bubble Sort	122,493	$5,00 \times 10^7$	0
Insertion Sort	0,045	$1,00 \times 10^4$	0
Merge Sort	1,107	$6,90 \times 10^4$	$6,90 \times 10^4$
Quick Sort	221,421	$5,00 \times 10^7$	$5,00 \times 10^7$

Fonte: elaborado pelos autores (2026)

Tabela 2B – Desempenho com Vetor Inversamente Ordenado

Algoritmo	Tempo (ms)	Comparações	Trocas
Bubble Sort	202,288	$5,00 \times 10^7$	$5,00 \times 10^7$
Insertion Sort	126,918	$5,00 \times 10^7$	$5,00 \times 10^7$
Merge Sort	1,387	$6,46 \times 10^4$	$6,46 \times 10^4$
Quick Sort	161,366	$5,00 \times 10^7$	$2,50 \times 10^7$

Fonte: elaborado pelos autores (2026)

Tabela 2C – Desempenho com Vetor em Ordem Aleatória

Algoritmo	Tempo (ms)	Comparações	Trocas
Bubble Sort	212,839	$5,00 \times 10^7$	$2,49 \times 10^7$
Insertion Sort	63,974	$2,49 \times 10^7$	$2,49 \times 10^7$
Merge Sort	1,801	$1,20 \times 10^5$	$1,20 \times 10^5$
Quick Sort	0,093	$1,52 \times 10^5$	$7,89 \times 10^4$

Fonte: elaborado pelos autores (2026)

As tabelas 2A, 2B e 2C exibem os resultados dos vetores com uma carga 10 vezes maior. O Insertion Sort ainda demonstrou bom desempenho no vetor ordenado, com apenas 0,045 ms e zero trocas, mas perdeu rendimento nos estados inverso (126,918 ms) e aleatório (63,974 ms). A classe que apresentou o maior tempo nos estados inverso e aleatório foi o Bubble Sort, superando os 202 ms. O Merge Sort continuou estável em todos os casos, mantendo-se na faixa previsível de 1,107 ms a 1,801 ms. O Quick Sort obteve desempenhos bem distintos: conquistou o menor tempo absoluto da escala no estado aleatório (0,093 ms), porém sofreu um severo declínio nos casos ordenado (221,421 ms) e inverso (161,366 ms), igualando-se à lentidão das abordagens quadráticas.

4.2 Cenário de Grande Escala (100.000 elementos)

Tabela 3A – Desempenho com Vetor Ordenado

Algoritmo	Tempo (ms)	Comparações	Trocas
Bubble Sort	12247,13	$5,00 \times 10^9$	0
Insertion Sort	0,318	$1,00 \times 10^4$	0
Merge Sort	11,8324	$8,54 \times 10^5$	$8,54 \times 10^5$
Quick Sort	20678,281	$5,00 \times 10^9$	$5,00 \times 10^9$

Fonte: elaborado pelos autores (2026)

Tabela 3B – Desempenho com Vetor Inversamente Ordenado

Algoritmo	Tempo (ms)	Comparações	Trocas
Bubble Sort	20157,936	$5,00 \times 10^9$	$5,00 \times 10^9$
Insertion Sort	12382,7831	$5,00 \times 10^9$	$5,00 \times 10^9$
Merge Sort	11,2724	$8,15 \times 10^5$	$8,15 \times 10^5$
Quick Sort	15389,939	$5,00 \times 10^9$	$2,50 \times 10^9$

Fonte: elaborado pelos autores (2026)

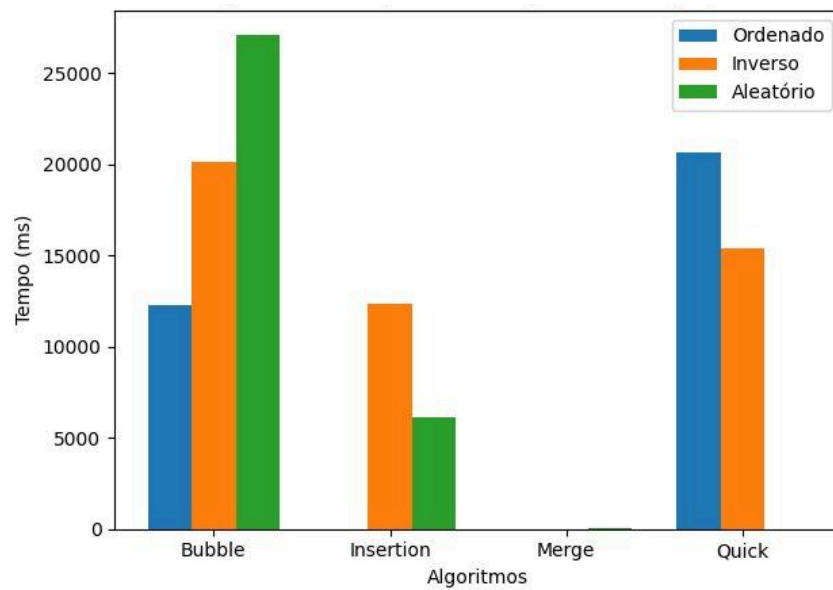
Tabela 3C – Desempenho com Vetor em Ordem Aleatória

Algoritmo	Tempo (ms)	Comparações	Trocas
Bubble Sort	27080,077	$5,00 \times 10^9$	$2,50 \times 10^9$
Insertion Sort	6122,449	$2,50 \times 10^9$	$2,50 \times 10^9$
Merge Sort	21,477	$1,54 \times 10^6$	$1,54 \times 10^6$
Quick Sort	12,612	$1,97 \times 10^6$	$1,08 \times 10^6$

Fonte: elaborado pelos autores (2026)

O panorama de grande escala com carga máxima de 100.000 elementos mostra que, no cenário ordenado, o Insertion Sort repetiu seu rendimento superior com incríveis 0,318 ms e nenhuma troca. No entanto, ele perdeu totalmente a eficiência nos casos aleatório e inverso, saltando para 6.122,449 ms e 12.382,783 ms. O Bubble Sort foi o mais lento desse cenário, levando mais de 27.080 ms no caso aleatório e operando na casa dos bilhões de comparações em todos os estados. O Merge Sort se manteve totalmente estável, resolvendo todas as frentes de forma equilibrada entre 11,272 ms e 21,477 ms. Por fim, o Quick Sort conquistou o melhor tempo no estado aleatório (12,612 ms), mas apresentou resultados péssimos nos casos ordenado (20.678,281 ms) e inverso (15.389,939 ms).

Gráfico 1 – Tempo x Cenário

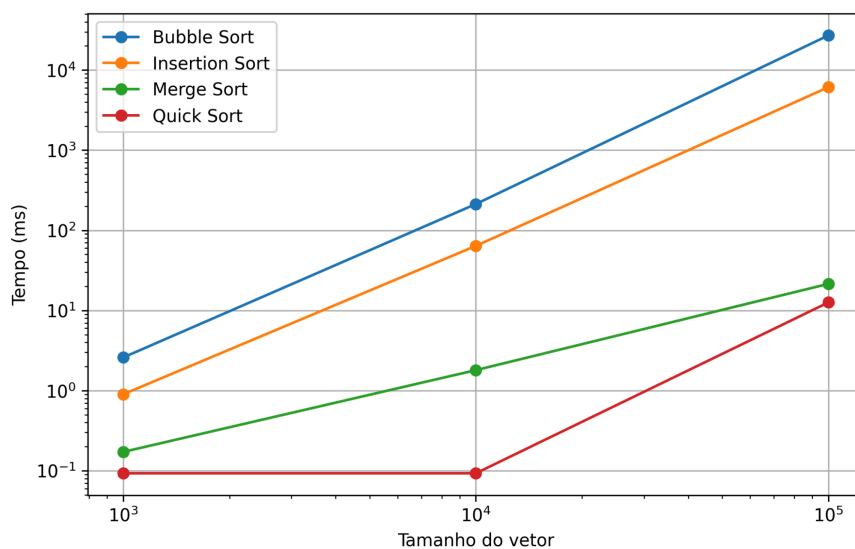


Fonte: elaborado pelos autores (2026)

O Gráfico 1 consolida visualmente o tempo de execução dos algoritmos em relação aos diferentes cenários estruturais analisados. A representação revela claramente a estabilidade do Merge Sort, que mantém seu desempenho praticamente inalterado independentemente do arranjo inicial dos dados. Em contraste, evidencia-se a forte volatilidade do Quick Sort, que atinge excelente velocidade no estado aleatório, mas sofre grande degradação nos casos ordenado e inverso.

4.3 Gráficos para Comparação

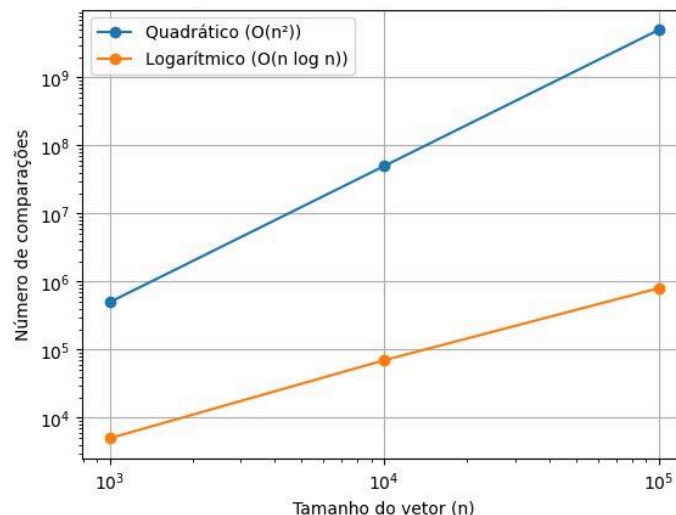
Gráfico 2 – Tempo × Tamanho do vetor (cenário aleatório)



Fonte: elaborado pelos autores (2026)

O Gráfico 2 expõe a evolução temporal dos algoritmos à medida que o volume de dados aumenta no cenário aleatório. A visualização mostra a forte divergência de escalabilidade entre as famílias de complexidade quadrática e logarítmica. Fica evidente que o Merge Sort e o Quick Sort suportam o crescimento da carga com grande eficiência, enquanto o Bubble Sort e o Insertion Sort perdem rendimento drasticamente.

Gráfico 3 – Comparações × Tamanho (quadráticos x logarítmicos)



Fonte: elaborado pelos autores (2026)

O Gráfico 3 foca no número de comparações lógicas realizadas pelas duas classes de complexidade. As linhas de tendência evidenciam a drástica diferença de ordens de grandeza no esforço computacional exigido conforme o tamanho do vetor cresce. Essa visualização comprova matematicamente que algoritmos logarítmicos executam uma fração mínima das operações exigidas pelos métodos quadráticos em conjuntos de dados maiores.

5. DISCUSSÃO

Os resultados obtidos confirmam as hipóteses iniciais acerca do comportamento dos algoritmos de ordenação em diferentes cenários de entrada. Nos vetores aleatórios de grande porte, especialmente no caso crítico de 100.000 elementos (Tabelas 3A, 3B e 3C), o Quick Sort e o Merge Sort apresentaram desempenho significativamente superior ao Bubble Sort e ao Insertion Sort (conforme ilustrado no Gráfico 2). Enquanto o Quick Sort liderou com o menor tempo de execução (12,612 ms), o Merge Sort demonstrou

estabilidade ao concluir em 21,477 ms. Esse comportamento evidencia na prática as vantagens da complexidade teórica $O(n \log n)$ em relação à $O(n^2)$ para grandes volumes de dados (vide Gráfico 3), corroborando as proposições clássicas de Cormen et al. (2012) sobre a eficiência de algoritmos de divisão e conquista. Ainda na análise deste cenário aleatório, Al-Amin et al. (2024) também encontraram em seus estudos que o Insertion Sort apresenta desempenho prático superior ao Bubble Sort, um achado que é diretamente corroborado por nossos dados experimentais.

Nos cenários em que os dados já estavam ordenados, o Insertion Sort demonstrou elevada eficiência adaptativa, registrando o menor tempo absoluto da rodada (0,318 ms no vetor de 100.000 elementos), realizando o número mínimo de comparações ($1,00 \times 10^4$) e nenhuma troca, conforme previsto teoricamente por Ziviani (2007). Em contrapartida, o Bubble Sort mostrou-se ineficiente ao manter um elevado número de operações mesmo quando o vetor já se encontrava em ordem (com tempo de 12.247,130 ms), confirmando sua rigidez estrutural e menor capacidade de adaptação a esse tipo de entrada.

Já nos vetores inversamente ordenados, o Bubble Sort e o Insertion Sort atingiram seus piores casos teóricos, apresentando custos operacionais idênticos e proibitivos, com exatamente 4.999.950.000 comparações e trocas no vetor de 100.000 elementos. O reflexo prático disso foi o disparo nos tempos de execução, atingindo 20.157,936 ms e 12.382,783 ms, respectivamente. Apesar do mesmo número de operações lógicas, a expressiva diferença no tempo absoluto pode ser atribuída ao maior custo computacional da função de troca (swap) no Bubble Sort em comparação ao simples deslocamento de memória (shift) realizado pelo Insertion Sort. De toda forma, esses achados confirmam que ambos os algoritmos tornam-se inadequados para conjuntos de dados extensos e reversos, em alinhamento com os alertas de Schildt (2006).

Um ponto crítico observado nos resultados foi o comportamento do Quick Sort no cenário ordenado e inversamente ordenado para 100.000 elementos, onde o algoritmo sofreu uma degradação severa de desempenho, levando 20.678,281 ms e 15.389,939 ms. Esse fenômeno ocorre devido à estratégia adotada na implementação, que estabelece o pivô fixo como o último elemento do subvetor, o que faz o algoritmo decair para uma complexidade quadrática $O(n^2)$ nesses cenários específicos, um comportamento amplamente documentado na literatura técnica (CORMEN et al., 2012).

Comparando-se todos os algoritmos sob uma perspectiva macro (ilustrada no Gráfico 1), observa-se que, embora o Quick Sort ofereça o melhor desempenho isolado para dados puramente aleatórios (12,612 ms e $1,97 \times 10^6$ comparações), o Merge Sort foi o único que manteve a consistência e a previsibilidade independentemente da organização inicial dos elementos, variando estavelmente entre 11,272 ms e 21,477 ms no maior vetor. No entanto, essa previsibilidade do Merge Sort exige concessões (trade-offs) que ficam evidentes nos dados: no cenário aleatório, o Merge Sort realizou $1,54 \times 10^6$ comparações, um volume superior ao exigido pelo Quick Sort para organizar a mesma entrada. Além disso, a consistência de tempo do Merge Sort tem como contrapeso o uso obrigatório de memória auxiliar $O(n)$, enquanto o Quick Sort opera de forma in-place. Portanto, a escolha evidencia um balanço claro: o Quick Sort otimiza o uso de memória e reduz comparações no caso médio, mas assume o risco de degradação severa de tempo (pior caso); já o Merge Sort garante imunidade ao arranjo inicial e preserva a estabilidade lógica das chaves, pagando o preço de um maior volume de comparações e maior consumo de RAM. Dessa forma, considerando simultaneamente o tempo de execução, o volume de operações e a imunidade ao arranjo inicial dos dados, o Merge Sort provou ser o algoritmo mais robusto e adequado ao cenário proposto.

6. CONSIDERAÇÕES FINAIS

Este estudo teve como objetivo central solucionar um problema de gargalo computacional no gerenciamento de vastos registros de uma biblioteca universitária, investigando qual algoritmo de ordenação apresentaria maior eficiência prática. Como demonstrado na seção de discussão, os testes experimentais permitiram validar com sucesso as três hipóteses norteadoras desta pesquisa. A análise comparativa demonstrou claramente que a adoção de algoritmos com complexidade quadrática, como Bubble Sort e Insertion Sort, é inviável para o cenário proposto devido ao custo operacional proibitivo em vetores de grande escala, especialmente quando os dados se encontram desordenados ou inversamente ordenados.

Com base nos dados experimentais, conclui-se que os algoritmos baseados na estratégia de divisão e conquista são a solução adequada. De forma específica, recomenda-se a implementação do Merge Sort no sistema da biblioteca. Embora o Quick Sort tenha apresentado o menor tempo absoluto em cenários puramente aleatórios, o Merge Sort

demonstrou total imunidade ao arranjo inicial dos dados, mantendo seu desempenho cravado na classe $O(n \log n)$ em todos os casos. Além disso, como ponto de destaque essencial para o cenário de uma biblioteca, reforça-se que o Merge Sort é um algoritmo estável. Essa característica de estabilidade representa uma vantagem prática crucial, pois permite a ordenação eficiente dos registros por múltiplos campos simultâneos e sucessivos, como ordenar por título do livro, por autor ou por data de empréstimo, preservando a ordem relativa dos elementos iguais estabelecida nas ordenações anteriores.

Apesar dos resultados robustos, este estudo apresenta limitações que devem ser consideradas. Os experimentos foram restritos à ordenação de vetores de números inteiros, não contemplando a manipulação de estruturas de dados mais complexas (como strings ou objetos compostos) que representam fidedignamente os títulos de livros e os cadastros de alunos. Ademais, o consumo de memória RAM não foi mensurado durante as execuções, o que é um fator relevante dado que o Merge Sort exige espaço auxiliar na memória para realizar as junções.

Como propostas para trabalhos futuros, sugere-se a instrumentação dos códigos para medir o custo de memória RAM de cada algoritmo, bem como a realização de novos testes utilizando tipos de dados textuais e estruturados. Adicionalmente, tendo em vista a severa degradação sofrida pelo Quick Sort em entradas ordenadas e inversas nesta pesquisa, recomenda-se investigar o impacto de estratégias mais robustas para a escolha do pivô, como a abordagem da mediana de três, a fim de mitigar a ocorrência do pior caso e otimizar o algoritmo para o sistema da biblioteca. Recomenda-se também a inclusão de outros algoritmos logarítmicos, como o Heap Sort, para expandir o escopo de comparação e refinamento do sistema da biblioteca.

REFERÊNCIAS

AL-AMIN, Ibrahim Muhammad; OKEYINKA, Aderemi Elisha; IBRAHIM, Abdullahi. **Comparative complexity study of bubble sort and insertion sort using java programming language: a review**. Journal of Science Innovation and Technology Research, v. 3, n. 9, p. 61–67, 2024.

ALENCAR, Amanda Laís. **QuickSort: Entendendo o Algoritmo**. Medium. Disponível em: <https://medium.com/@amndalsr/quicksort-entendendo-o-algoritmo-b31b0807fd95>. Acesso em: 7 maio 2026.

BACKES, André. **Estruturas de Dados Descomplicada em C**. Rio de Janeiro: Elsevier, 2016.

BHARGAVA, Aditya Y. **Entendendo Algoritmos: Um guia ilustrado para programadores e outros curiosos**. 1. ed. São Paulo: Novatec, 2017.

CORMEN, Thomas Henry; LEISERSON, Charles Eric; RIVEST, Ronald Linn; STEIN, Clifford. **Algoritmos: Teoria e Prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.

KARTIK. **Learn DSA in C: Master Data Structures and Algorithms Using C**, Geeks For Geeks. Disponível em: <https://www.geeksforgeeks.org/c/learn-dsa-in-c>. Acesso em: 16 abril 2026.

SCHILDT, Herbert. **C Completo e Total**. 3. ed.. São Paulo: Makron Books, 2006.

SODHI, Tarundeep Singh; KAUR, Surmeet; KAUR, Snehdeep. **Enhanced insertion sort algorithm**. International Journal of Computer Applications, v. 64, n. 21, p. 35–39, fev. 2013.

ZIVIANI, Nivio. **Projeto de algoritmos: com implementações em Java e C++**. São Paulo: Cengage Learning, 2007.